

What Broke, and How It Got Fixed

Incident log of an audited audit — every failure, its root cause, its fix, and how the fix was verified · rounds 1–3: 2026-08-31 (15:20–18:53) · documentation & release-gate pass: 2026-09-04 · canonical chronological source: `submission/NOTEBOOK.md` (rounds 1–4)

Why this document exists. The assignment scores unverified claims negatively and auto-fails fabricated evidence. So this log is deliberately honest: the work was AI-heavy under human direction, things broke in all three rounds — including in the fixes themselves — and the interesting part is that the *process* caught every one: measure-before-claim, numbers-under-test, and finally a nine-agent adversarial review of the submission itself. Nothing below is dramatized; each row traces to a diff and, where numeric, to a `verify.py` assertion.

Round 1 (15:20–15:46) — building the audit, catching my own hands

Incident	Root cause	Fix	Verified by
FLORES extractor died with <code>KeyError</code>	Assumed tarball member paths; real members carry a <code>./</code> prefix	Listed members first, corrected the path	5 languages × 1012 lines extracted (extended to 7 in round 2); SHA-256 pins recorded
NFC hypothesis wrong — twice	Predicted nukta decomposition would shift toy numbers (it didn't: 0.00%); then assumed no-op generalizes (it doesn't: 93 hin / 609 ben FLORES lines change)	Kept the intern's NFC call, upgraded verdict from "harmless" to "actively useful", measured its effect ($\leq +0.51\%$, ben)	E4 all-zero deltas; NFC table in <code>results_tables.md</code>
<code>lower()</code> bug direction assumed backwards	Intuition said the bug made Hindi look worse; measurement says it inflates only English fertility	Claim written from the measurement, not the intuition	E2: hin bit-identical, ratio +2.92%
Two rounded-table arithmetic errors in draft docs (18.55 vs 18.48; +23% vs +29%)	Deriving ratios from 2-decimal display values	Re-derived every quoted figure from full-precision CSV before finalizing	Later formalized as <code>verify.py</code> assertions
Notebook timestamps ran ahead of the wall clock	Reconstructed a plausible timeline instead of recording the real one	Checked <code>Get-Date</code> , corrected every stamp, logged the slip in the notebook itself	NOTEBOOK entry 15:46

Round 2 (~16:00–16:30) — after the operator rejected draft 1 ("not professional enterprise level")

Incident	Root cause	Fix	Verified by
The A4 memo asserted "romanized Hindi"	The audit broke its own evidence rule in its own memo	Built <code>romanization.py</code> ; measured: 2.27× on gpt2 (not ~1×) and <i>worse than native</i> on	4 live <code>verify.py</code> checks (both tokenizers, native + romanized)

tokenizes like English" — unmeasured		o200k for Hindi (1.88 vs 1.57); round-3 pipeline fixes (candra, danda) refined these to the current 2.25 / 1.87	
Published "35 checks" while the harness ran 40	Hand-typed count	Harness now counts and prints itself (ALL n NUMBERS VERIFIED); docs quote the printed value. (The bulk find- replace that fixed the documents also overwrote the notebook's own historical "35" — caught by the round-4 release gate, restored in NOTEBOOK round 4)	Current output: ALL 50
Figure label collisions (2)	Annotation placed without rendering	Rendered, looked, moved/removed labels	Re-rendered PNGs

Round 3 (~16:45–18:53) — after rejection 2: the audit gets audited

Escalation: a deterministic multi-agent workflow — **9 adversarial reviewer lenses** → **merge** → **two-lens verification per finding** (59 agents, 784 tool calls — scale logged in NOTEBOOK round 4). 50 raw findings → 31 merged → **30 confirmed, 1 refuted**. The highlights, all fixed and re-verified the same evening:

Incident	Root cause	Fix	Verified by
The 128k-vocab trap — missed by me, caught by the panel. All "current stack costs 7.4×" pricing assumed v0's gpt2 is the serving tokenizer; the spec says vocab = 128k	Inherited v0's framing without reconciling it against v0's own spec sheet	Every coupling claim hedged and bracketed; table relabelled "v0's tokenizer"; measuring FLM-4B's real tokenizer promoted to a recommendation; the one tokenizer-independent claim (3584-token English content can't fit in Hindi) kept, with its justification (min premium $1.16 > 1.14$)	REPORT_v1 §4; memo/partC hedges
My own logic turned on me: "three independent goodput derivations (± 0.4 tok/s)" — two were algebraically identical; the third cancelled the latency data out	Exactly the false-confirmation pattern the audit's C2 finding condemns in the intern	Rewritten: definition + genuinely independent latency route ($n/itl = 249.8$; roofline) + a labelled identity	Renamed verify checks; capacity.md B3
Tamil romanization partly an artifact: transliterator invents aspirates Tamil script cannot express (+24% inflation); raw	Scheme output never validated end-to-end	Candra mapping + danda normalization; residual-script counting added (now 0 everywhere); Tamil published as a measured range (2.37–3.09×); docstring example	Regenerated romanization_output.md; residual check in verify.py

Devanagari candra vowels survived in 190/1012 "romanized" hin lines; danda became " "; the docstring example didn't match real output		generated from a live run; reviewer probes reproduced before quoting	
Prose-number drift, as a class: "7.0× per code point" (a toy-corpus number pasted into a FLORES claim; truth 7.46×), "9.6–15.5×" over a list whose minimum is 7.9×, "~13× (Tamil)" vs the tables' 14.7×, "±1.2% CIs" vs measured worst 1.28%, a "–14%/+37%" leftover	Prose quoting numbers no script emits	All corrected <i>and</i> all asserted — harness grew 40 → 50 checks, including a CI-bound assertion computed from results.csv itself	verify.py (ALL 50)
Part C's gates were tie-inflated: a do-nothing SFT would pass "win-or-tie ≥70%"; LoRA runtime undercounted (loss tokens, not full sequences)	Metric not tested against the null intervention	Tie-proof gates (win share of non-ties + tie-rate cap + absolute rubric rate); 90M tokens → 5 h/run, ~65 runs (conclusion holds at 3× margin)	partC/memo.md
Deliverability was silently broken: nothing committed to git (a "git link" submission would ship empty); <code>core.autocrlf</code> would rewrite the hash-pinned corpus on any Windows clone; four scripts hardcoded paths; the bootstrap hardcoded n=10 six times (a pasted line would leave the CI unchanged); AI_USAGE ended in a template placeholder	Working-tree development; environment assumptions never tested against the delivery path	<code>_paths.py</code> resolver + <code>STARTER_KIT</code> ; CLI overrides (<code>--corpus/--hin/--log</code>); derived sizes; root <code>.gitattributes</code> * <code>-text</code> + <code>.gitignore</code> ; CI workflow; placeholder → honest defense-prep checklist; everything committed and pushed	Live smoke tests (pasted file moved the CI [5.37,6.43]→[5.32,6.37]); push f12c3ae..de0749e; CI on GitHub
Two breakages <i>during the fixing</i> : hardcoded "ben +1.2%" into a generator minutes after fixing that exact class; a	Habit outliving its correction; side-effectful smoke test	Note derived from computed deltas; canonical output regenerated	Final full verify run: ALL 50

smoke test overwrote the canonical experiment output			
--	--	--	--

Round 4 (2026-09-04) — the documentation pass gets the same treatment

This document and its two siblings were hand-authored, so a three-agent release gate audited them the way round 3 audited the submission: a fresh-clone verification (passed all six steps — verify green from a clone, corpus hashes intact through checkout, overrides live, regenerated outputs byte-identical), a forensic docs-vs-ground-truth number audit, and a link/staleness sweep. The gate found 13 issues in the drafts — including a rounded "40×" where the machine output says ~38×, dates conflating the two working days, and the discovery that a round-3 bulk find-replace had silently rewritten the notebook's own historical record (the exact opposite of a lab notebook's job). All fixed; the corrections and the workflow-scale facts are recorded in NOTEBOOK round 4 so this page's every number is repo-traceable. GitHub Actions is green on every pushed commit.

The operator's role (the human, stated honestly)

This project was AI-executed under human direction — and saying so precisely is what makes it an interview asset rather than a liability. The operator:

- **Set and enforced the quality bar** — rejected draft 1 ("professional enterprise level is what I want") and draft 2 ("not as per standards"), refusing to accept work that a staff panel would puncture; both rejections are what triggered rounds 2 and 3, where the most important defects (the unmeasured romanization claim, the 128k trap, the circular corroboration) were found.
- **Chose the escalation strategy** — authorized the multi-agent adversarial review instead of another incremental polish pass; that decision is directly responsible for 30 verified findings.
- **Made the delivery decisions** — corpus/scope direction ("architecture and intro files first"), when to commit, and the push to GitHub.
- **Owns the defense**: the pre-submission checklist in `AI_USAGE.md` lists the re-derivations to perform personally (verify run, one live toggle, B1 on paper, one bench row by hand) — the repo is built so each is one command, and disagreement with any AI phrasing is itself evidence of ownership.

How to present this in an interview

- **The claim to make**: "I directed an AI-heavy engineering workflow to evidence-grade output: every published number is machine-asserted (50 checks, in CI), every claimed bug has a one-toggle experiment, and when the output wasn't good enough I escalated to an adversarial multi-agent review of the work itself — which found 30 real defects, including two my own methodology explicitly warns about. Here is the incident log."
- **The claim not to make**: hand-authorship of code that was AI-written, or debugging sessions that didn't happen. The repo's `AI_USAGE.md` is explicit, the git history is public, and the defense is live — honesty here is both mandatory and, given the process above, the stronger story.
- **The one-liner**: the audit's method (measure, verify, adversarially review) was strong enough that it caught its own author — twice by hand, thirty times by machine.

Sources: submission/NOTEBOOK.md rounds 1–4 (chronology, workflow scale, restored incidents) · submission/AI_USAGE.md (AI failure incidents 1–7) · git history github.com/Manureddy148/AI_Task2 · compiled 2026-09-04.